Упражнение 4 Графични трансформации. Транслация, ротация, скалиране.

Графичните трансформации са математически операции, които променят позицията, ориентацията и размера на обекти в двумерно (2D) или тримерно (3D) пространство. Те се използват широко в компютърната графика, игрите, анимациите и симулациите.

Целта на упражнението е запознаване с основните графични трансформации транслация (преместване), ротация (завъртане) и скалиране (мащабиране). Разглежда се как тези трансформации могат да бъдат представени чрез матрично умножение и как се прилагат в компютърната графика.

Основни моменти:

- Да си припомним концепцията за хомогенни координати;
- Използване на матрици за трансформация;
- Прилагане на последователност от трансформации върху обекти;
- Реализация на основните трансформации в графична среда.

1. Хомогенни координати

В евклидовата геометрия точките се представят с Декартови координати, но в компютърната графика често се използват хомогенни координати за по-удобно представяне на трансформации.

Хомогенните координати добавят допълнителен компонент към стандартните декартови координати. Вместо да представяме точка в 2D с (x,y), използваме тройката реални числа(x,y,w). За 3D пространството добавяме още една координата: (x,y,z,w).

Хомогенните координати позволяват еднообразно представяне на всички линейни и афинни трансформации като матрично умножение. В класическите декартови координати някои трансформации като транслация не могат да се представят само чрез линейно умножение. При нея се изисква векторно събиране:

$$(x', y') = (x + tx, y + ty)$$

В хомогенни координати точката (x,y) се разширява до (x,y,1), а транслацията става матрично умножение:

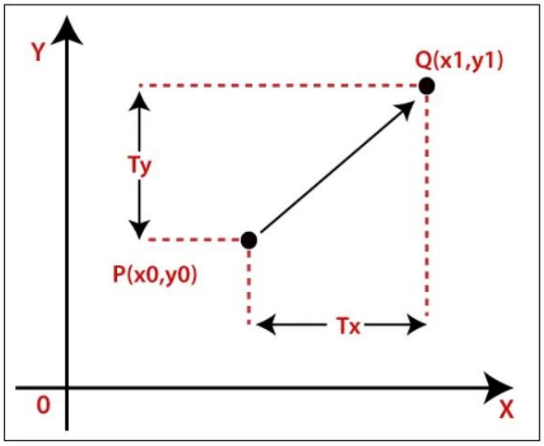
$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & tx \\ 0 & 1 & ty \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Така всички геометрични трансформации (включително транслация) могат да се представят унифицирано като матрични операции. Скалиране, ротация, срязване и транслация могат да бъдат изразени чрез 3×3 матрици (в 2D) или 4×4 матрици (в 3D). Това опростява комбинирането на трансформации чрез умножение на матрици.

Хомогенните координати играят ключова роля и в представянето на проективни трансформации, тъй като позволяват представянето им като матрично умножение.

2. Графични трансформации – транслация

Транслацията представлява преместване на обект в пространството, без да се променят неговата форма, размер или ориентация.



Ако точка P(x,y) се премести с вектор (t_x,t_y), новите ѝ координати са:

$$x' = x + t_x, \quad y' = y + t_y$$

Транформационната матрица ще има следния вид:

$$T = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix}$$

Резултатът от транслацията може да бъде установен след матрично умножение:

$$P'_h = T \cdot P_h = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} x + t_x \\ y + t_y \\ 1 \end{bmatrix}$$

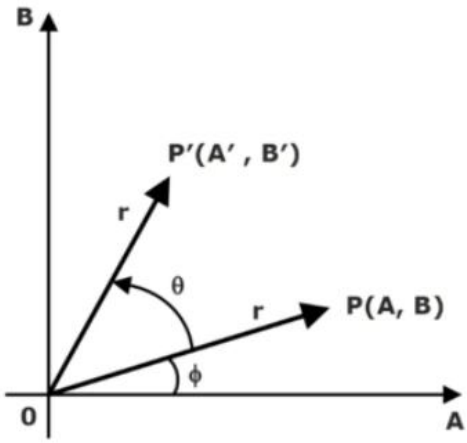
При 3D координатна система транформационната матрица добавя допълнителна компонента t_z:

$$T = \begin{bmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$P'_h = T \cdot P_h = \begin{bmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} = \begin{bmatrix} x + t_x \\ y + t_y \\ z + t_z \\ 1 \end{bmatrix}$$

3. Графични трансформации – ротация

Ротацията завърта обект около определена ос (в 3D) или точка (в 2D), без да променя неговия размер.



При завъртане на точка в 2D с координати (x,y) с ъгъл θ, новите координати са:

$$x' = x\cos\theta - y\sin\theta$$

$$y' = x\sin\theta + y\cos\theta$$

Ротационната матрица има следния вид:

$$R_\theta = \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Ротация в 3D – завъртане около осите.

Матрица за ротация около оста x:

$$R_x = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos\theta & -\sin\theta & 0 \\ 0 & \sin\theta & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Матрица за ротация около оста y:

$$R_y = \begin{bmatrix} \cos\theta & 0 & \sin\theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin\theta & 0 & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Матрица за ротация около оста z:

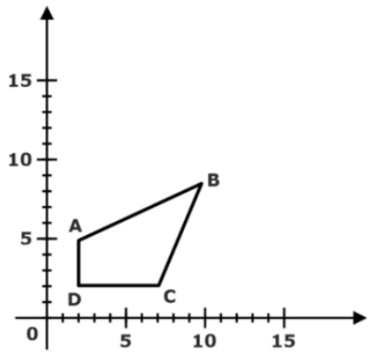
$$R_z = \begin{bmatrix} \cos\theta & -\sin\theta & 0 & 0 \\ \sin\theta & \cos\theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

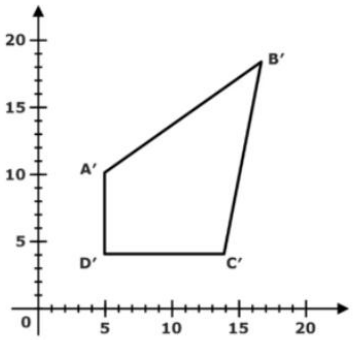
$$P'_h = R_x \cdot P_h = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos\theta & -\sin\theta & 0 \\ 0 & \sin\theta & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

4. Графични трансформации – скалиране

Скалирането променя размера на обекта по дадена ос или и в двете направления.

За да извършим 2D мащабиране, ние използваме коефициенти на мащабиране: S_x за оста x и S_y за оста y. Тези фактори определят колко всяка координата трябва да бъде мащабирана по съответната ос. Ако коефициентът на мащабиране (S_X и S_Y) е по-голям от 1, обектът се увеличава и се отдалечава от началото. Коефициент на мащабиране 1 оставя обекта непроменен, докато коефициент на мащабиране по-малък от 1 свива обекта и го премества по-близо до началото.





Ако мащабираме точка (x,y) със скаларни коефициенти s_x и s_y, новите координати са:

$$x' = s_x \cdot x$$

$$y' = s_y \cdot y$$

Матрици за мащабиране в 2D/ 3D

$$S = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$S = \begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Новите координати на скалираната точка (3D) ще се определят от произведението:

$$P'_h = S \cdot P_h = \begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

5. Прилагане на последователност от трансформации върху обекти

В компютърната графика и 3D моделирането често се налага да прилагаме не една, а няколко трансформации последователно върху даден обект. Тези трансформации могат да включват транслация (преместване),

ротация (завъртане) и скалиране (мащабиране). В хомогенни координати всички трансформации могат да бъдат представени чрез матрично умножение. Ако имаме последователност от трансформации $T_1, T_2, \ldots, T_n$, тогава новото положение на обекта се получава чрез:

$$P'_h = T_n \cdot T_{n-1} \cdot \ldots \cdot T_1 \cdot P_h$$

където:

- **Ph** е векторът на обекта в хомогенни координати,
- **Ti** са матриците за трансформации.

Редът, в който прилагаме трансформациите, е изключително важен, защото матричното умножение не е комутативно. Ако например трябва да извършим 1. мащабиране 2. ротация 3. транслация, то крайният резултат ще бъде различен от 1.транслация 2. ротация 3. мащабиране. Обикновено се следва реда скалиране (задаваме правилния размер на обекта), ротация (завъртане около оста, преди да го преместим) и накрая транслация (преместване в желаната позиция).

6. Реализация на основните трансформации в графична среда

Тъй като на този етап ще включим към проекта работа с транформационна матрица, е необходимо в класа ShaderProgram да добавим още няколко метода, с помощта на които можем да я подадем към вертексния шейдър, както и други типове данни (float, матрица, вектор и др.).

```
public void setUniform(String name, Matrix4f matrix) {
    int location = glGetUniformLocation(programId, name);
    if (location != -1) {
        FloatBuffer buffer = BufferUtils.createFloatBuffer(16);
        matrix.get(buffer);
        glUniformMatrix4fv(location, false, buffer);
    }
}

public void setUniform(String name, float[] values) {
    int location = glGetUniformLocation(programId, name);
```

```
    if (location != -1) {
        glUniform1fv(location, values);
    }
}

public void setUniform(String name, int value) {
    int location = glGetUniformLocation(programId, name);
    if (location != -1) {
        glUniform1i(location, value);
    }
}

public void setUniform(String name, float value) {
    int location = glGetUniformLocation(programId, name);
    if (location != -1) {
        glUniform1f(location, value);
    }
}
```

След като имаме начин да подаваме трансформационната матрица, е необходимо да създадем класа, в който ще бъдат съхранени данните за тази матрица. За целта създайте класа Transform в пакета scene.

```
package bg.tu_varna.sit.gs.scene;

import org.joml.Matrix4f;
import org.joml.Vector3f;

import static org.joml.Math.toRadians;

public class Transform {
```

```
  private Vector3f translation = new Vector3f();

  private Vector3f rotation = new Vector3f();

  private Vector3f scale    = new Vector3f(1, 1, 1);

  private Vector3f pivot = new Vector3f();


  private final Matrix4f modelMatrix = new Matrix4f();

  private boolean transformChanged = true;

}
```

В този клас са поместени векторите за транслация, ротация и скалиране на даден обект. Освен тях ние имаме и точката спрямо която трябва да бъдат извършени дадените трансформации, и по спецефично ротазцията, а именно точката pivot. Накрая имаме и самата матрица, в която ще се изчисляват всичките трансформации, които трябва да бъдат приложени към обекта, както и булева стойност, чрез която ще изчисляваме тази матрица само ако има някаква промяна в нея.

Следва да добавим методите, чрез които ще променяме стойностите на различните трансформации.

```
  public void markTransformChanged() {

    transformChanged = true;

  }


  public void setTranslation(Vector3f translation) {

    this.translation = translation;

    markTransformChanged();

  }


  public void setRotation(Vector3f rotation) {

    this.rotation = rotation;

  }
```

```
public void setScale(Vector3f scale) {
    this.scale = scale;
    markTransformChanged();
}

public void setPivot(Vector3f pivot) {
    this.pivot = pivot;
    markTransformChanged();
}

public void rotateY(float angle){
    rotation.y += toRadians(angle);
    markTransformChanged();
}

public void rotateX(float angle){

    rotation.x += toRadians(angle);
    markTransformChanged();
}

public void rotateZ(float angle){
    rotation.z += toRadians(angle);
    markTransformChanged();
}
```

И накрая ще добавим и метода, чрез който ще извличаме цялостната трансформационна матрица.

```
public Matrix4f getModelMatrix() {
    if (transformChanged) {
        modelMatrix.identity()
            .translate(translation)
            .translate(pivot)
            .rotateZ(rotation.z)
            .rotateY(rotation.y)
            .rotateX(rotation.x)
            .translate(-pivot.x, -pivot.y, -pivot.z)
            .scale(scale);

        transformChanged = false;
    }

    return modelMatrix;
}
```

При извличавено на крайната трансформационна матрица, първо проверяваме дали е необходимо тя да бъде изчислена наново, спрямо булевата стойност. Ако е необходимо да бъде изчислена наново извършваме операциите в следия ред:

1. скалираме обекта
2. транслираме обекта, така че центъра му да е нашата pivot точка
3. извършваме ротация по оста X, Y, Z
4. транслираме обекта, обратното действие на точка 2
5. транслираме обекта в сцената

Тези действия се записват в обратен ред, за да може да се приложат правилно всичките трансформации.

След като имаме обекта Transfrom, трябва да го добавим като поле към класа SceneObject, за да могат всички обекти, които добавяме към сцената, да

имат матрица за трансформации и всеки отделен обект да може да се манипулира по отделно.

```
private Transform transform = new Transform();
```

След това добавете и getter и setter метод за този атрибут.

За да можем да използваме матрицата за трансформации и да я подаваме към вертексния шейдър, използвайки методите, които добавихме към ShaderProgram, ще променим логиката за рендериране в класа OpenGLRenderer.

```
@Override
public void render(Scene scene) {
  shader.use();

  for (SceneObject sceneObject : scene.getSceneObjects()) {
    Transform transform = sceneObject.getTransform();
    shader.setUniform("modelMatrix", transform.getModelMatrix());
    sceneObject.render(shader);
  }
}
```

По този начин преди да се рендерира всеки един от обектите на сцената се извлича неговият обект Transfrom и използвайки метода getModelMatrix() се подава матрицата с трансформации към шейдъра.

Вертексният шейдър ще придобие следния вид:

```
#version 460 core
layout(location = 0) in vec3 aPos;
layout(location = 1) in vec3 aColor;
out vec3 vertexColor;
uniform mat4 modelMatrix;

void main() {
gl_Position = modelMatrix*vec4(aPos, 1.0);
```

```
        vertexColor = aColor;

        }
```

Накрая нека да променим къщичката, като я смалим, завъртим на 90 градуса и я преместим надясно. За целта в класа SceneBuilder след като създадем новия обект къща, ще променим нейната трансформация по следния начин.

```
SceneObject house = new House();

house.getTransform().setScale(new Vector3f(0.3f, 0.3f, 1f));

house.getTransform().rotateZ(90);

house.getTransform().setTranslation(new Vector3f(0.5f, 0f, 0f));

scene.addSceneObject(house);
```

7. Създаване на движение

Следващият код създава анимира къщата в OpenGL, като използва униформена трансформационна матрица (modelMatrix) за транслация. Основната идея е да актуализира позицията на къщата в метода update() преди всеки кадър, което създава ефект на движение.

В класа House допълнете следните полета за задаване на начална позиция и скорост на движение:

```
private float translationX = 0.0f;

private float speed = 0.005f;
```

В метода update() добавете код, описващ изменението на координатите преди всеки кадър. При достигане на границите за движение, се обръща неговата посока.

```
  @Override

  public void update() {

    translationX += speed;

    if (translationX > 0.5f || translationX < -0.5f) {

      speed = -speed;
```

```
    }


    getTransform().setTranslation(new Vector3f(translationX, 0f, 0f));
}
```

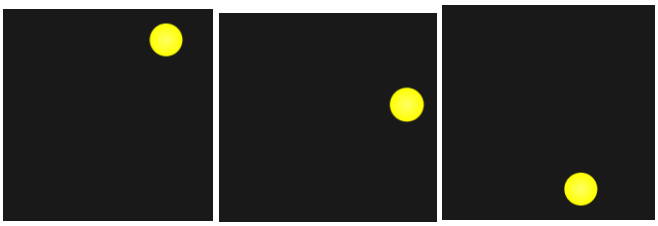
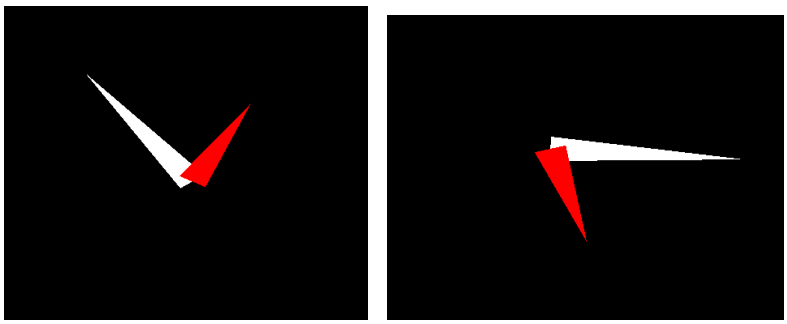
Задачи за самостоятелна работа

1. Тествайте примерите, посочени по-горе. Направете така, че къщата да се движи по диагонал.

2. Изчислете новите координати на триъгълник с върхове A(-0.5, -0,5, 0.0), B(0.5, -0,5, 0.0) и C(0.0, 0.5, 0.0) след завъртане на 30 градуса около оста z. Реализирайте трансформацията в графична среда.

3. Изчислете новите координати на триъгълник с върхове A(-0.5, -0,5, 0.0), B(0.5, -0,5, 0.0) и C(0.0, 0.5, 0.0) след скалиране Sx=1.2, Sy=0.7 и транслиране tx=0.3, ty=0.4. Реализирайте трансформацията в графична среда.

4. Рендирайте следния сценарий: имате изрисуван кръг, който обикаля около стените на прозореца. Достигайки даден лимит на движение в посоката си надясно, той се ориентира надолу. Достигайки лимит на движение надолу, се ориентира наляво и т.н.



5. Създайте часовник с въртящи се стрелки. Обърнете внимание на факта, че ротацията с положителен ъгъл в openGl е по посока, обратна на часовниковата стрелка.